Dukaan

Project Profile Document AMD DevMaster Hackathon, Track 1: Development of Multimodal Content Creation Tools

Application	Dukaan
Team	himanshu748 (solo)
Hardware	AMD Radeon PRO, gfx1100 (RDNA 3), 48 GB VRAM, 48 CUs
Stack	ROCm 7.2.4, PyTorch 2.10.0+rocm7.2.4, ComfyUI 0.25.0 as a library
Models	LTX-2.3 22B (audio-video diffusion), Gemma 3 12B (text encoder)
Licence	Apache-2.0

1. Project background

A shopkeeper with a phone can photograph a product in ten seconds. Turning that photograph into something postable is the part that does not happen. It needs a background that is not a countertop, three different crops for three different places, and type that is legible at thumbnail size. That is an afternoon in a design tool, or a few hundred rupees to someone who owns one, per product, every time the stock changes.

The tools that promise to close this gap mostly generate the product too. That is the wrong trade. A seller cannot post a picture of a bangle that is not the bangle they will ship. Whatever the model does, the object has to survive it.

Dukaan takes one photograph and returns a set of finished creatives plus a short clip with sound, generated on a Radeon GPU. It ranks selected frames against the input plate for reference consistency, asks the seller to review the product, and composites the seller's words rather than asking a model to draw them.

2. Target users and application scenarios

Primary user. A single-person or family retail business with between ten and a few hundred SKUs, selling through Instagram, WhatsApp Business, or an Indian marketplace listing. They photograph stock themselves. They have no designer and no subscription budget, and their catalogue turns over faster than they can commission artwork for it.

Scenarios

Scenario	What Dukaan produces
New stock arrives, needs a feed post today	1024x1024 square with headline, price line and contact
Festival push, same products, seasonal look	Same photos re-run under the `festive` style, no re-shoot
Story or reel slot	1024x1820 still plus an H.264/AAC MP4 with generated ambient audio
Marketplace or shop-page header	1820x1024 banner
Price change	`dukaan relabel` recomposes the same stills with new type, no GPU
Wrong moment in the shot	The browser UI's scrubber rebuilds from any of the 49 frames, no GPU

Why the constraints matter. Sellers photograph against whatever is behind the counter, so the background is rarely one flat colour. They type prices and phone numbers that must appear exactly as typed. They post the same product to three places with three aspect ratios. Each of these drove a specific design decision in section 3.

3. System architecture

```
  laptop                                    Radeon instance
----------                                ---------------------
photo
  |
  +-- cutout (CPU, quadratic background fit)
  |
  +-- plate: product on the style wash --->  phase 1  text encoder (23 GB)
  |                                                   |
  |                                                   v  conditioning, bf16
  |                                          phase 2  checkpoint (43 GB)
  |                                                   |
  |                                                   v
  |                                          49 frames + stereo wav
  |   <----------- one tar over the JupyterLab API ---+
  |
  +-- pick one still per format (reference consistency + sharpness)
  +-- reshape by reflection to each aspect
  +-- composite headline, subline, contact strip
  |
  v
square / story / banner / clip.mp4 / preview.gif / wav / manifest.json
```

Transport. The cloud exposes JupyterLab over HTTPS and nothing else: no ssh, no scp. Its REST API carries all three needs, so `dukaan/instance.py` uses the contents API for files, the kernels API for a process, and one websocket for that process's stdout. Long jobs are started under `setsid` so a culled kernel cannot take them down, and progress is read back from their logs.

Split of work. Everything cheap and deterministic stays on the CPU on purpose. The cutout is a least-squares fit; the still selection combines a small structural and colour descriptor with Laplacian variance; the reshaping is an index map; the type is a font. Spending GPU credits on any of them would be waste. The GPU does the one thing with no CPU equivalent, and it does it **once per pack** rather than once per output.

3.1 Two surfaces, one pipeline

`dukaan serve` puts the same pipeline behind a browser form: photo, headline, price, look, button. It calls `build_pack`, the path the CLI uses, so the two surfaces cannot disagree about what the tool does. The rules list a web UI and a CLI plus demo workflow as valid delivery forms; this ships both.

The scrubber is the part that only this architecture allows. Because the only generative model on the instance is a video model, a pack is not three renders, it is one generated scene with three moments lifted out of it. The other 46 frames are already on disk and already paid for, so choosing a different one rebuilds the creative on the CPU in milliseconds and reports **0 seconds of GPU**. A tool built on an image model cannot offer that, because each candidate would be another generation.

Measured through the UI against the live Radeon: 94.4 s for 49 frames at 768x768, then rebuilding the square from moment 30 instead of moment 1 at no GPU cost. It is also the honest answer to a real failure mode: the model occasionally leaves an artefact in a frame, and the seller can simply pick another moment instead of paying for a re-roll.

4. Models and algorithms

4.1 LTX-2.3, and why the pipeline is shaped around it

The instance carries exactly one generative model: `ltx-2.3-22b-dev`, an audio-video diffusion model, plus its Gemma 3 12B text encoder, a spatial upscaler and two LoRAs. There is no image-only model on disk.

Fetching one is awkward rather than impossible, and the reason is worth recording for anyone else building on this hardware. The instance's outbound network cannot reach `huggingface.co` or `github.com` directly. PyPI, `raw.githubusercontent.com`, `hf-mirror.com` and `modelscope.cn` all answer normally, so weights can be pulled through a mirror with `HF_ENDPOINT=https://hf-mirror.com`.

Dukaan still uses LTX alone, by choice rather than by constraint. The 43 GB checkpoint is already on disk, and a video model turns out to be the better primitive for this job: if the generator produces moving scenes, stills are lifted out of a scene rather than generated one at a time, so three formats can look like three different shots for the cost of one generation. Adding a second model would also mean a second 20-plus GB resident in a container that already cannot hold the pair it ships with, which is the constraint section 5 is about.

The graph mirrors the workflow that ships with the template, so the semantics match what the ComfyUI editor would run:

```
LTXAVTextEncoderLoader -> CLIPTextEncode x2 -> LTXVConditioning
CheckpointLoaderSimple -> LoraLoaderModelOnly (distilled, 0.5)
LTXVPreprocess -> LTXVImgToVideoInplace -> LTXVConcatAVLatent
  -> CFGGuider + SamplerCustomAdvanced (euler, 8-step manual sigmas)
  -> LTXVSeparateAVLatent -> VAEDecodeTiled  (video)
                          -> LTXVAudioVAEDecode (audio)
```

The sigma schedule and the 0.5 LoRA strength are taken from the shipped workflow, not tuned: the distilled LoRA is trained for that schedule.

`strength=0.7` on `LTXVImgToVideoInplace` is deliberately kept low to reduce product drift. It cannot guarantee that a logo, engraving or stone stays exact, which is why the post-generation screening and human review remain required.

4.2 Background removal by fitting, not sampling

Backdrops are rarely one colour. A counter shot shades off as window light falls away; catalogue photography grades the sweep on purpose. Differencing against a single sampled colour keeps whichever half drifted furthest.

Dukaan fits a quadratic surface per channel to the image's border ring and differences against the fit:

- basis `[u, v, u², v², uv, 1]` on normalised coordinates, solved by least squares
- refit once with the worst quarter of residuals dropped, so a product running off the frame edge cannot drag the surface towards itself
- threshold, feather, use as alpha

A plane was tried first. It cleared the top-to-bottom wash and left an elliptical pool of backdrop exactly where the product sits, because catalogue lighting pools rather than ramps. The quadratic basis is what fixed it.

Per-pixel thresholding is not enough on its own, and the two ways it fails were both visible in an earlier version of the gallery. A light product on a lit sweep reads partly as backdrop, so the middle of it comes back full of holes: on the silver bracelet that was 14,497 pixels of the band. And a lobe of backdrop the fit cannot account for survives as a patch stuck to the product. Both are obvious once whole regions are considered instead of pixels, which is what `dukaan/regions.py` does: drop blobs far smaller than the product, then close background pockets that cannot reach the frame edge.

The size limit on that second step matters more than it looks. A first version closed every enclosed pocket and nearly doubled the gilt bangles' mask, from 14.7% of the frame to 28.2%, because **a bangle's interior is real backdrop and filling it turns two rings into two discs**. A teapot handle has the same property. Only pockets smaller than 2% of the product's own area are closed now.

A known limitation. One of the sample photographs, the blue-and-white porcelain vase, has a vignette *behind* the object. No fit to the border ring can see it, and the lobe it leaves is genuinely fused to the vase, so no region filter can separate the two. A morphological opening was measured as a way out and rejected: at radius 6 the lobe still held 33,110 pixels and at radius 12 still 15,403, while by radius 10 the brass ewer had lost 1.9% of frame, which is its spout and handle. Refitting the surface iteratively on non-product pixels was also measured, and made the lobe marginally worse while shrinking the silver bracelet's mask by 1.9%. Paying real product detail for a defect that survives anyway is a bad trade, so the opening ships disabled with its measurements next to it, and that photograph is not shown as if it were good output. The honest fix is a matting model, which section 8 lists as the next step.

4.3 Still selection

Frames are split into as many contiguous windows as there are formats. Inside each window, a lightweight reference-consistency heuristic compares central edge structure and colour against the input plate at several zooms. That signal receives most of the score, while Laplacian variance prevents a blurred frame from winning. Frame 0 is never eligible: it is the plate the model was handed.

The score is a screening and ranking signal, not an identity metric. Every selected score, threshold and warning is written to the manifest, and the UI lets the seller inspect or replace a selected frame without another GPU call. `dukaan catalogue` also writes `catalogue-audit.json` with attempted, completed, failed and review-required counts so a phone-photo field evaluation can report failures rather than hiding them.

4.4 Reshaping without cropping the product

A square frame becoming a 9:16 story cannot be cover-cropped, because the product lives in the sides that would be cut. The frame is scaled to fit whole and the leftover bands are filled from its own content, **mirrored**, with the join blurred. Clamping the edge row was tried first and drew the bokeh background into long horizontal streaks.

4.5 Type is composited, never generated

Image models garble text, and a seller's price and phone number are the two things that must be exactly right. Headlines shrink to fit rather than truncate, because the words are the seller's. An early run also reproduced a craft channel's watermark across the bottom of a frame, so the negative prompt now names the shapes the model reaches for.

5. Adaptation for AMD Radeon GPU and ROCm

5.1 The instance cannot run the template it ships with

LTX-2.3 diffusion checkpoint	43 GB
Gemma 3 12B text encoder	23 GB
Total to load	**66 GB**
Container cap, `/sys/fs/cgroup/memory.max`	**55 GB**

ComfyUI's server holds every model a graph touches for the life of the process. Loading both trips the cap and the platform restarts the container mid-prompt. JupyterLab returns with zero kernels, port 8188 stops answering, and anything written outside the persistent volume is gone. It presents as a network fault.

This was diagnosed by watching `memory.current` during a load and reading `/proc/1`'s start time across a failure, which showed the container itself being replaced rather than the process dying.

5.2 The fix: two processes that never overlap

`scripts/radeon_ltx.py` imports ComfyUI as a library rather than starting its server, and runs the graph in two phases:

```
phase 1   load the text encoder, encode the prompts, write conditioning
          to disk, exit so the process gives its RAM back
phase 2   load the diffusion checkpoint, read the conditioning back,
          sample, write frames and audio, exit
```

Peak becomes `max(43, 23)` instead of `43 + 23`. Measured on the box:

phase	peak container RAM	wall
encode	35.4 GB	33 s
sample	51.2 GB	55 s
stock template, one process	trips 55 GB, container restarts	n/a

5.3 Batching, the one lever that changes the shape of the cost

Every other setting trades quality for time. Batching does not: it removes work that was never necessary.

Loading the checkpoint costs about 17 seconds and the text encoder about 9, and neither depends on how much you then generate. A shop packing fifty items one photo at a time pays that toll fifty times. Dukaan's `catalogue` command encodes every prompt against one text-encoder load, then samples every plate against one checkpoint load:

```
[checkpoint-loaded]   17.1s
[lora+audio-vae]      17.9s
[sampled 1/3]         63.7s     first product: 45.8 s
[sampled 2/3]         98.3s     second:        34.6 s
[sampled 3/3]        121.5s     third:         23.2 s
[models-evicted]     125.3s
```

Two effects compound. The load is paid once instead of three times, and per-product time then falls by half across the batch as the GPU warms, for identical work. Three products cost 125 s batched against roughly 186 s run separately.

The ordering this requires is worth stating: every product is **sampled** before anything is **decoded**. The VAE decode needs the 22B transformer out of VRAM, and evicting it between products would mean reloading 43 GB per product, which is the entire cost the batch exists to avoid. Latents are small, so holding all of them until the transformer is gone is cheap.

5.4 Three further adaptations

bf16 conditioning. The conditioning is read back while the 43 GB checkpoint is already resident, so every megabyte comes off the remaining 4 GB of headroom. Writing it in bf16 halved it, 1470 MB to 735 MB. It fell under one megabyte once the correct AV encoder replaced a plain `CLIPLoader`, which is also how the wrong encoder was caught: LTX-2.3's text embeddings carry both a video and an audio stream, and the plain loader produced a 4-D tensor the model rejected.

VRAM eviction before the VAE decode. ComfyUI's executor evicts models between nodes. Calling node classes directly skips that, so the 22B transformer was still resident when the VAE ran and the decode failed with 2.13 GB free out of 47.98. Calling `model_management.unload_all_models()` before the decode drops the container from 51.2 GB to 12.5 GB and frees the VRAM the decode needs. Tiled decode (512 px spatial, 32-frame temporal chunks) bounds the peak.

`torch.inference_mode()` **around the whole phase.** The LTX VAE updates the sampler's output in place, and torch refuses that across the inference-mode boundary. ComfyUI wraps every graph this way; calling nodes directly has to do the same.

`PYTORCH_ALLOC_CONF=expandable_segments:True` is set for the sampling phase to keep HIP allocator fragmentation from re-introducing the OOM.

5.5 How the memory numbers were measured

Reading `memory.current` when a run finishes reports about 12 GB, because the models have already been evicted by then. Quoting that as the pipeline's footprint would be flattering and wrong, and it is the number an obvious implementation reports: the first version of the benchmark did exactly this and claimed 11.8 GB for a pipeline whose real peak is over 50.

The kernel's own `memory.peak` is not usable either. Resetting it needs a write this kernel rejects, so it reports the high-water mark since the container booted rather than since the run started. Dukaan therefore samples `memory.current` on a thread every 200 ms and keeps the maximum, which is per-run by construction and costs one file read per sample.

5.6 Measured results

setting	output	GPU time	peak container RAM	MPx-frames/s
512x512, 25 frames	512x512	45.5 s	49.8 GB	0.14
768x768, 25 frames	768x768	54.6 s	49.9 GB	0.27
768x768, 49 frames	768x768	70.7 s	49.9 GB	0.41
768x768, 49 frames, refined	**1536x1536**	176.1 s	49.9 GB	0.66
3 products, one batch	768x768	**134.3 s**	49.9 GB	vs 212.1 s separately

Reproduce with `dukaan bench <photo>`; the raw JSON is in `bench-results/`. Throughput rises with the size of the job because the 13.8 s model load is fixed, which is the whole argument for the batch.

A single pack end to end, including the plate upload, both model loads, sampling, decode and pulling 49 frames plus a wav back over the tunnel, is about 2 m 20 s wall clock. Before frames were fetched as one tar it was 3 m 34 s, and the tunnel reset the connection twice mid-run.

Batching measured on the same settings as row 3: 13.8 s of model load paid once, then 36.3 s, 28.0 s and 26.6 s for three identical products as the GPU warms.

5.7 Two template defects worth reporting upstream

- The LTX notebook's own cell invokes `start_comfyui_with_rc_tunnel.sh`; the file on disk is `start_comfyui_with_tunnel.sh`. Run All fails.
- ComfyUI runs from `/opt/venv`, not the system python, so starting it by hand the obvious way gives `No module named 'sqlalchemy'`.

6. Verification

`dukaan doctor` reports what a run would actually use before it costs anything:

```
video_encoder: ffmpeg
arch: gfx1100
vram_gb: 48.0
compute_units: 48
torch: 2.10.0+rocm7.2.4.git3d3aa833
card_model: 0x744b
container_ram_cap_gb: 55.0
runner: yes
ready
```

`rocm-smi` cannot reach libdrm inside this container and torch returns an empty device name, so the architecture, PCI model id and CU count stand in. They are what the driver will actually report.

The automated suite runs with no GPU. It covers cases that were genuinely wrong at some point: phone EXIF orientation, safe output paths, a graded backdrop, a product touching the frame edge, reference-aware selection, MP4 muxing, catalogue audit records and reflection versus streaking when a frame is reshaped.

Without `DUKAAN_INSTANCE` the whole pipeline runs against a CPU mock and writes real files, so the tool can be inspected before any GPU spend. The mock is never a fallback: if an instance is configured and fails, the error surfaces.

7. Results

Nine creatives and three clip previews with audio, all generated on the Radeon, are in `docs/gallery.md`. New packs also export a ready-to-post H.264/AAC MP4. Source, tests and the runner are in the repository.